# On the Road of Computing

(конспективные заметки, сжато
излагающие компьютерный курс,
каким он должен быть по моему мнению)

Stanislav Gubin
anno Domini 0x7DF-0x7E0

# Оглавление

**1 Дискретная математика** **4**
1.1 Компьютерная арифметика . . . . . 4
1.1.1 Системы счисления . . . . . 4
1.1.2 Отрицательные двоичные числа . . . . . 5
1.1.3 Двоичная арифметика . . . . . 5
1.1.4 Плавающая точка . . . . . 6
1.1.5 Восьмеричная и шестнадцатеричная системы счисления . . 7
1.1.6 Перевод между системами счисления . . . . . 7
1.2 Логика высказываний и булева алгебра . . . . . 8
1.2.1 Логика высказываний . . . . . 8
1.2.2 Булева алгебра . . . . . 9
1.2.3 ДНФ . . . . . 10
1.3 Теория множеств, отношения и функции . . . . . 10
1.3.1 Множества . . . . . 10
1.3.2 Отношения . . . . . 12
1.3.3 Функции . . . . . 13
1.4 Конечные автоматы и тьюринг-полные формальные системы . . . . 14
1.4.1 Конечные автоматы . . . . . 14
1.4.2 Тьюринг-полные формальные системы . . . . . 15
1.5 Структуры данных . . . . . 16
1.5.1 Массивы . . . . . 17
1.5.2 Связные списки . . . . . 17
1.5.3 Деревья . . . . . 18
1.5.4 Хэш-таблицы . . . . . 19
1.5.5 Очереди . . . . . 20
1.5.6 Стеки . . . . . 21
1.6 Вопросы . . . . . 22

# Введение

[тут будет эпиграф]

[не готово, весь текст введения ниже — черновой]

## О чем эта книга

[потом будет]

## Структура книги

[черновой вариант, пока есть только глава 1 (неполная)]

1. Дискретная математика — логика, булева алгебра, теория множеств, теория графов (потом), комбинаторика (потом), автоматы и модели вычислителей, алгоритмы (потом) и структуры данных

2. Цифровая логика — логические элементы, цифровые схемы, устройство регистров, памяти, АЛУ

3. Внутренний мир процессора — устройство процессора, процесс выполнения команды, ускорение работы ЦП: кэш, предвыборка, изменение последовательности команд, прогнозирование переходов, скрытые регистры, конвейеризация

4. Машинный язык — виды команд процессора, режимы адресации, адресное пространство программы, архитектура x86

5. Операционные системы — управление памятью (виртуальная страничная в частности), управление процессами, управление устройствами, файловые системы, системные вызовы

6. «Жизнь» пользовательской программы поверх вышеописанного — написание, компиляция, компоновка, загрузка на исполнение, динамическая компоновка

7. Основной инструмент: UNIX-подобная ОС (Linux, *BSD etc.), философия, устройство файловой системы, командный интерпретатор bash, редактор Emacs (альтернативно, только я его не знаю — Vi/Vim), управление проектами при помощи make, управление программами (поиск, установка, обновление etc.) на примере системы Portage дистрибутива Gentoo Linux, верстка документации при помощи LaTeX

8. Язык программирования C — типизация, операторы, процедуры, указатели, структуры и битовые поля, стандартная библиотека, «хороший» стиль программирования

9. Ассемблер x86, программирование в командах процессоров x86 (x86, MMX, SSE etc.)

10. Системное программирование в UNIX/Linux

# Глава 1

# Дискретная математика

[тут будет эпиграф]

## 1.1 Компьютерная арифметика

### 1.1.1 Системы счисления

В быту мы привыкли использовать десятичную систему счисления, что исторически обусловлено числом пальцем на руках homo sapiens. Важным здесь есть то, что эта дело исключительно опыта и привычки, ничего фундаментального в таком выборе нет, т.е. десятичная система ничем не «лучше» других. В случае компьютеров она, более того, оказывается невероятно неудобной. Во всех компьютерах (не считая отдельных экзотичных моделей на заре компьютерной эпохи, среди которых были и троичные, и десятичные монстры) используется *двоичная система* счисления, оперирующая только цифрами 0 и 1. Это сделано только по одной причине — цифровая информация хранится в уровнях напряжения (высокий сигнал соответствует 1, низкий — 0, или наоборот) и техника легче всего воспринимает ее, когда диапазонов как можно меньше (одного, однако, мало, т.к. такой сигнал не несет информации): в случае использования привычной 10-ной системы пришлось бы делить тот же максимальный уровень сигнала на 10 маленьких отрезков и было бы тяжело понять, находясь на границе, является ли текущий сигнал цифрой 3 или 4, в случае двоичной это максимально легко.
Общая формула получения значения n-разрядного числа в позиционной системе счисления с основанием b следующая:

$$a_{n-1}a_{n-2}...a_1a_0 = \sum_{i=0}^{n-1}(a_ib^i)$$

В случае двоичной системы очевидно, она вырождается в:

$$a_{n-1}a_{n-2}...a_1a_0 = \sum_{i=0}^{n-1}(a_i 2^i)$$

причем $a_i$ это 0 или 1 для $0 \leq i \leq n-1$, $a_i$ называется *битом*. Число комбинаций, допускаемых n-битным двоичным числом, равно $2^n$, от 0 до $2^n - 1$.

### 1.1.2 Отрицательные двоичные числа

Для хранения отрицательных значений большинство компьютеров используют т.н. систему с дополнением до двух. Справедливости ради стоит сказать, что существует как минимум 4 системы:

- система со знаком. Старший (здесь и далее под ним понимается крайний левый бит) бит определяет знак, остальные - модуль. Таким образом мы получаем диапазон $[-2^{n-1}+1; 2^{n-1}-1]$, что очень плохо, поскольку у нас есть два разных представления нуля;
- система с дополнением до 1. Получается инвертированием (заменой на противоположные) всех битов числа. Остается тот же диапазон и, следовательно, та же проблема, что у предыдущей;
- система с дополнением до 2. Получается инвертированием всех битов с последующим прибавлением (см. ниже) 1, что спасает от двойного нуля, но обеспечивает несимметричный диапазон $[-2^{n-1}; 2^{n-1}-1]$, что, однако, не так страшно, как трата комбинации впустую;
- система со смещением на $2^{n-1}$. Число воспринимается как положительное (т.е. $[0; 2^n - 1]$) $-2^{n-1}$, т.е. мы получаем такой же диапазон, как в предыдущей, но процесс получения процессор осуществляет немного дольше, чем просто инверсия с прибавлением 1.

Как видим, лучше всего себя показывает система с дополнением до 2 (она же «дополнительный код»), потому она и признана общемировой уже многие десятилетия.

### 1.1.3 Двоичная арифметика

- Сложение двоичных чисел принципиально не отличается от десятичных, за исключением того, что перенос происходит при получении 2 (т.к. такой цифры нет), а не 10.

- Вычитание реализуется путем того же сложения с числом, представленным в дополнительном коде.

- Умножение реализуется многократным сложением либо (для целых чисел) битовым сдвигом на $n$ бит влево в случае умножения на $2^n$, что рекомендуется использовать разработчикам процессоров и компиляторов (да и всем остальным программистам тоже, т.к. где гарантия, что компилятор совершит нужную оптимизацию? Язык C предоставляет множество встроенных средств для подобных оптимизаций).

- Деление реализуется многократным вычитанием с наращиванием счетчика, но в случае целых чисел справедливы те же битовые сдвиги (но вправо) для степеней двойки.

### 1.1.4 Плавающая точка

Ранее я говорил только о целых числах, но как быть с вещественными (такими как 5.124, например)?
Мы можем хранить как целые числа целую и дробную часть по-отдельности, но, очевидно, такое решение будет дегенеративно по своей сути, т.к. будет порождать числа разной длины, либо же (если мы решим, скажем, выделить 30 бит как максимальную длину целой части и 20 как дробной) будет порождать чрезмерно большие числа, там где точность на таком уровне не нужна (5.3 например).
Потому используется *экспоненциальная нотация*, когда вещественное число представляется как произведение *мантиссы* (задающей точность) и степени (задающей величину), которая, в свою очередь, определяется через *экспоненту*, т.е. показатель этой степени, а основание степени равно основанию системы счисления. Для двоичной системы вещественное число $N = s * m * 2^n$, где $m$ — мантисса, $n$ — экспонента, $s$ — знак (1 или -1).
Компьютерная реализация такой концепции описана в стандарте IEEE 754. Согласно нему, все вещественные числа представлены тремя группами - одинарной (`float` в C), двойной (`double` в C) и повышенной (`long double` в некоторых компиляторах C) точности. Для каждой из них последовательность бит числа состоит из знакового бита $s$, $n$ бит экспоненты $N$ и $m$ бит мантиссы $M$ ($m >> n$).
Битовые комбинации для каждого класса разбиты на 4 группы (+ учет знака, он в таблице не показан), см. табл. 1.1.
Отсюда видно, что, к примеру, модули 32-битовых `float`-чисел имеют диапазон $[2^{1-2^{8-1}+1}(1+0) = 2^{-126}; 2^{254-2^{8-1}+1}(1+\sim 1) =\sim 2^{128}]$ обычных значений, но могут принимать и очень маленькие значения, минимальное из которых $= 2^{-2^{8-1}+1}2^{-23} = 2^{-150}$.

| Группа | Представление | Значение |
|---|---|---|
| Обычные числа | $N$ в $[1; 2^n - 2]$ | $2^{N-2^{n-1}+1}(1+M)$ |
| Округ 0 | $N = 0$ | $2^{-2^{n-1}+1}M$ |
| Бесконечность | $N = 2^n - 1, M = 2^m - 1$ | $\infty$ |
| Неопределенность | $N = 2^n - 1, M \neq 2^m - 1$ | NaN |

Таблица 1.1: Числа с плавающей точкой в IEEE754

### 1.1.5 Восьмеричная и шестнадцатеричная системы счисления

В программировании очень популярны также системы счисления с основаниями 8 и 16 (т.к. оба числа есть степени двойки), позволяющие более кратко записывать значения битовых последовательностей (к примеру, адресов памяти компьютера).
В восьмеричной системе используются цифры от 0 до 7, в шестнадцатеричной - от 0 до 10 плюс a,b,c,d,e,f (обычно как в большом, так и в малом регистре) для цифр со значениями с 11 по 15 соответственно.
В начале шестнадцатеричных чисел для указания на их природу часто ставят 0x (к примеру, 0xFF это 255), восьмеричных - 0, впрочем, такой подход принят по крайней мере в языке C, в разных языках и книгах может быть по-разному.

### 1.1.6 Перевод между системами счисления

Правило перевода числа между двумя любыми системами счисления следует из определения — достаточно подбирать наибольшие степени целевой системы, не большие текущего остатка (изначально он равен всему числу), вычитать их из него, записав цифру, показывающую, сколько степеней было вычтено, на соответствующей позиции, и продолжать это до тех пор, пока остаток не станет нулем - тогда все оставшиеся разряды справа также нулевые.
Есть и другой способ, заключающийся в делении на основание целевой системы счисления, делая следующий делитель равным текущему частному и запоминая остатки, до тех пор, пока частное не станет нулем. Тогда последовательность из всех остатков в обратном порядке даст искомое представление.
Перевод между двоичной и восьмеричной или двоичной и шестнадцатеричной же системами (который используется постоянно и нас особенно потому интересует) куда проще и осуществляется заменой блоков из 3 или 4 бит на восьмеричную или шестнадцатеричную цифру эквивалентного значения соответственно (и в обратную сторону), поскольку и 8, и 16 - степени 2. Если в блоке меньше 3 (или 4 для шестнадцатеричной) бит, слева дописывается соответствующее число нулей.

## 1.2 Логика высказываний и булева алгебра

### 1.2.1 Логика высказываний

Логика нулевого порядка (логика высказываний) является одним из базовых разделов дискретной математики, на котором основана логика предикатов.
Во многих курсах логика высказываний отделяется от булевой алгебры, описываемой намного позже, я же объединю соответствующие блоки в одну секцию, поскольку операторы, законы и константы булевой алгебры находятся в очевидном соответствии с таковыми логики высказываний (и также теории множеств, но ее стоит вынести отдельно ввиду концептуальной разницы ее понятий с логическими, а также ввиду того, что в ее блоке будут рассматриваться еще и отношения и функции).
*Высказывание* можно понимать как утверждение, которое может быть либо *истинным*, либо *ложным*, причем истину и ложь следует понимать как аксиоматические, базовые и противоположные друг другу понятия (можно заменить их на 1 и 0 или что-нибудь подобное, не нужно привязываться к используемым в бытовой речи словам и понятиям).
Из элементарных высказываний (неразбиваемых на более простые) можно строить более сложные путем *логических операторов*, являющихся $n$-арными функциями высказываний ($n$ обычно равно 1 или 2), результирующее значение, очевидно, всецело определяется значениями операндов.
Важную роль играет приоритет операторов, для повышения которого используются круглые скобки. К примеру, просьба *«принесите мне чай или кофе и пиццу»* можно понимать по-разному (см. табл. 1.2 на с. 9): если приоритет логического *«или»* больше, то просьба удовлетворяется следующими вариантами - *«чай и пицца», «кофе и пицца», «чай и кофе и пицца»*, если же выше приоритет *«и»*, то удовлетворительными являются *«чай», «кофе и пицца», «чай и кофе и пицца»*, как видим, последовательности не совпадают из-за неоднозначности приоритета.
И если в быту мы, скорее всего, решим, что пицца имеется в виду как обязательный элемент (не эквивалентный напиткам), а из напитков нужен только один (т.е. подсознательно получим выбор из *«чай и пицца», «кофе и пицца»*, восприняв «или» как «исключающее или»), то это никак не касается формальной логики и того, как бы подобную конструкцию воспринимал компьютер. Впрочем, обычно приоритет «и» более высокий, чем у «или», но это лишь примитивный пример — существует огромное множество случаев, где без скобок не обойтись (не считая таких нотаций, как *обратная польская запись*).
Чтобы не повторяться, я не буду здесь строить таблицы истинности сложных высказываний, так как они по сути своей идентичны таблицам соответствующих булевых функций, которые далее и рассмотрим.

### 1.2.2 Булева алгебра

*Булева алгебра* — множество (см. раздел 1.3.1 на с. 10, для понимания данного материала понятие множества не обязательно), на котором определены константы 1 и 0, а также определенный набор функций $f_i : B^n \to B$. Здесь мы видим явное сходство 1 с «истиной» логики, а 0 — с «ложью» (или наоборот, что не важно при условии перестройки таблиц истинности и соблюдении такой замены далее).
Нас, как компьютерщиков, интересует не связь между теорией множеств, логикой и булевой алгеброй (более интересная математикам), а логические операции над двоичными числами, совершаемые компьютером. Потому мы рассмотрим наиболее распространенный в нашей области набор булевых функций:

| A | B | eq A | not A | A or B | A and B | A xor B | A nor B | A nand B | A nxor B |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 |

Таблица 1.2: Некоторые логические операции

Понятно, что «or» в данном случае это то же «или» и т.д. Также понятно, что многие столбцы таблицы излишни, т.к. A nand B по сути дела то же, что not(A and B) и т.п. Более того, только через функции not, and и or можно реализовать любую булеву функцию (см. раздел 1.2.3 на с. 10), а поскольку каждая из них может быть реализована на nand либо на nor, получаем, что nand либо nor достаточно для получения любой булевой функции.
Доказывать эквивалентность функций можно не только с помощью таблиц истинности, но и заменой символов в формулах, при помощи т.н. законов булевой алгебры (ранее, во избежание путаницы с арифметическими операторами, я использовал «and», «or», «not» и т.п., теперь же, краткости ради, они будут заменены на $*, +, !$ соответственно):

| **Название** | **and-формула** | **or-формула** |
|---|---|---|
| Тождество | 1A=A | 0+A=A |
| Закон нуля | 0A=0 | 1+A=1 |
| Идемпотентность | AA=A | A+A=A |
| Инверсия | A!A=0 | A+!A=1 |
| Коммутативность | AB=BA | A+B=B+A |
| Ассоциативность | A(BC)=(AB)C | A+(B+C)=(A+B)+C |
| Дистрибутивность | A+BC=(A+B)(A+C) | A(B+C)=AB+AC |
| Поглощение | A(A+B)=A | A+AB=A |
| Законы де Моргана | !(AB)=!A+!B | !(A+B)=!A!B |

Таблица 1.3: Законы булевой алгебры

Отметьте, что or-формы получаются из and-форм путем взаимной замены $*$ на $+$ и 0 на 1 (и наоборот). Это позволяет использовать только одну форму, при каждом доказательстве получая соответствующее выражение второй.

### 1.2.3 ДНФ

Для получения универсальной формулы любой функции по ее таблице истинности используется т.н. *дизъюнктивная нормальная форма.* Она получается как дизъюнкция (+) элементарных конъюнкций (*) элементов (если 1) или их инверсий (если 0) тех строк, в которых функция равна 1.
К примеру, рассмотрим функцию Y, описанную таблицей:

| A | B | C | Y |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Ее ДНФ будет равна !A!BC+A!B!C+AB!C+ABC.
Если же нужно получить максимально упрощенную функцию по данной таблице истинности (т.е. такую, формула которой была бы как можно короче), нужно сначала получить ее ДНФ, а затем упростить ДНФ, пользуясь либо законами булевой алгебры (однако этот путь весьма туманен, т.к. нет гарантии, что мы получим максимальное упрощение), либо специальными алгоритмами, такими как старый *алгоритм карт Карно* или *алгоритм Куэйна-МакКласки*, на которых, в силу конспективности пособия, я останавливаться не буду.

## 1.3 Теория множеств, отношения и функции

### 1.3.1 Множества

Теория множеств — одно из фундаментальных оснований всей математики.
*Множество* можно понимать как некую совокупность объектов — т.е. это понятие является аксиоматическим, не выражаемым формально через другие.
Для множеств справедливы следующие свойства (здесь и далее $\vee$ обозначает логическое or, $\wedge$ — and, % — остаток от деления, «:» в определениях множеств —

«такие, что», $\neg$ — логическое not, $\Leftrightarrow$ — «эквивалентно», $\Rightarrow$ — «следует, что»):

- Множество задается перечислением всех элементов либо описанием закона, которому подчиняются все элементы.
  К примеру, выражения $A = \{3, 5, 7, ...129, 131, 0.3, 0.31, 0.32, ...0.35, \pi\}$ и $A = \{x : (2 < x \leq 131 \wedge x \in \mathbb{N} \wedge x\%2 = 1) \vee (0.3 \leq x \leq 0.35 \wedge 100x \in \mathbb{N}) \vee x = \pi\}$ задают одно и то же множество;

- Порядок расположения элементов роли не играет.
  К примеру, выражение $A = \{0.3, 0.31, ...0.35, 3, 5, 7, ..., 129, 131, \pi\}$ задает то же множество $A$ из примера выше;

- Повторения элементов недопустимы;

- Любое множество $A$ обладает *мощностью* $C$ — т.е. определенным количеством элементов. Это обозначается как $|A| = C$.
  Для *конечных* множеств мощность равна конечному числу, для бесконечных — $\infty$, причем бесконечные мощности также могут быть различны между собой: у *счетных* множеств (т.е. таких, из которых существует биекция в натуральный ряд (см. раздел 1.3.3 на с. 13) $\infty$ элементов всегда меньше, чем у *несчетных*, к примеру, оба множества натуральных и вещественных чисел являются бесконечными, но мощность второго больше, т.е. между каждыми двумя его элементами $a$ и $b$ можно поместить элемент $c = \frac{a+b}{2}$, т.е. действительные числа образуют *континуум*, в отличие от натуральных.
  Для любого бесконечного множества получить множество с иной мощностью (но тоже бесконечно большой) можно бесконечным числом способов;

- *Подмножеством* данного множества $A$ есть такое множество $B$, что все элементы $B$ есть элементы $A$, это обозначается как $B \subset A$. Иначе говоря, $B \subset A \Leftrightarrow (x \in B \Rightarrow x \in A)$;

- *Пустое множество* $\emptyset = \{\}$ — это множество, не содержащее элементов. Для любого множества $A$ $\emptyset \subset A$;

- *Универсум* $U$ это множество такое, что любое рассматриваемое множество есть его подмножество: $A \subset U$;

- Над множествами можно производить определенный набор операций:

  - *Объединение* множеств $\cup$: $A \cup B = \{x : x \in A \vee x \in B\}$.
    Как видим, оно семантически тождественно логическому «or»;
  - *Пересечение* множеств $\cap$: $A \cap B = \{x : x \in A \wedge x \in B\}$.
    Здесь аналогичное соответствие с «and»;

– *Разность* множеств $\setminus$: $A \setminus B = \{x : x \in A \wedge \neg(x \in B)\}$;

- Для множеств справедливы следующие законы (отметьте соответствие между ними и законами булевой алгебры (см. табл. 1.3 на с. 9)):

| Закон | $\cup$-форма | $\cap$-форма |
|---|---|---|
| Коммутативность | $A \cup B = B \cup A$ | $A \cap B = B \cap A$ |
| Ассоциативность | $A \cup (B \cup C) = (A \cup B) \cup C$ | $A \cap (B \cap C) = (A \cap B) \cap C$ |
| Дистрибутивность | $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ | $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ |
| Свойства $\emptyset, U$ | $A \cup U = U$<br>$A \cup \emptyset = A$<br>$A \cup \neg A = U$ | $A \cap \emptyset = \emptyset$<br>$A \cap U = A$<br>$A \cap \neg A = \emptyset$ |
| Идемпотентность | $A \cup A = A$ | $A \cap A = A$ |
| Законы де Моргана | $\neg(A \cup B) = \neg A \cap \neg B$ | $\neg(A \cap B) = \neg A \cup \neg B$ |
| Поглощение | $A \cup (A \cap B) = A$ | $A \cap (A \cup B) = A$ |

Таблица 1.4: Основные законы алгебры множеств

- *Декартовым произведением* множеств $A \times B$ есть множество всех пар, первые элементы которых принадлежат $A$, вторые — $B$(причем пары являются упорядоченными), иначе говоря: $A \times B = \{(a, b) : a \in A \wedge b \in B\}$ . Понятно, что $|A \times B| = |A||B|$;

- *Булеаном* множества $A$ называется множество всех подмножеств данного множества: $B = \{X : X \subset A\}$. Если $B$ — булеан $A$, то $|B| = 2^{|A|}$(это легко понять, представив битовую строку длины $|A|$, где каждый бит = 1, если соответствующий элемент входит в очередной элемент B, и 0 в противном случае. Число битовых комбинаций составит $2^{|A|}$.

### 1.3.2 Отношения

*Отношением* из множества $A$ в множество $B$ есть какое-то подмножество $A \times B$. К примеру, $A = \{x : x$ —«операционная система»$\}, B = \{y : y$ —«процессорная архитектура»$\}$, тогда отношение «платформы» (понимаемаемое как совокупность процессорной архитектуры и работающей поверх нее ОС) из $A$ в $B$ может иметь следующий вид: $\{(Gentoo\ Linux, x86_64), (System\ V, DEC\ PDP), (Solaris, SPARC)\}$.
Отношение может быть задано как обычное множество или как матрица битов, где 1 стоит на пересечении тех элементов, что входят в отношение, 0 — всех остальных.
К примеру, ранее описанное отношение «платформы» может иметь такую матрицу:

| | **DEC PDP** | **SPARC** | **x86_64** | **x86** |
|---|---|---|---|---|
| **Gentoo Linux** | 0 | 0 | 1 | 0 |
| **System V** | 1 | 0 | 0 | 0 |
| **Solaris** | 0 | 1 | 0 | 0 |

Таблица 1.5: Матрица отношения «платформы»

Отношения — невероятно важное понятие, в частности такая их разновидность, как *функции*.

### 1.3.3 Функции

Математическое понимание функции не следует путать с компьютерным. В программировании функция (или, более верно, *процедура*) — это последовательность команд, хранимая в памяти, вызываемая по своему адресу из другой процедуры, после завершения возвращая ей управление. Мы еще поговорим о таком представлении и достаточно подробно обсудим его реализацию, но сейчас мы говорим исключительно о математических абстракциях, в которых нет такого понятия, как время или порядок исполнения.
*Функция* — как обычно говорят, «зависимость, при которой каждому значению аргумента соответствует только одно значение функции», более формально, это отношение из $A$ в $B$, когда *для каждого элемента* $A$ существует *только одно* значение из $B$. Это обозначается как: $f : A \to B$ или (чаще встречаемая запись) $y = f(x)$, где $x \in A, y \in B$, при этом $A$ называют *областью определения*, $B$ — *областью потенциальных значений*.
Интересны отдельные разновидности функций:

- *Инъективная функция* — такая, что не существует двух различных аргументов, которые бы дали одинаковое значение функции (т.е. она *всегда изменяется*), иначе говоря: $f(x) = f(y) \Leftrightarrow x = y$;
- *Сюръективная функция* — такая, что для каждого элемента $B$ найдется соответствующий аргумент из $A$, или: $(\forall y \in B)\exists x \in A : y = f(x)$;
- *Биективная функция* — одновременно инъективна и сюръективна, она же *взаимно однозначное соответствие*.

Я не буду далее останавливаться на том, какие бывают виды функциональных зависимостей, на их графиках или таких сущностях, как производные или интегралы, поскольку это нашего предмета непосредственно не касается. Более того, слишком частое контактирование с понятиями матанализа может сбить с толку начинающего компьютерщика, которому следует сразу уяснить: *в цифровых компьютерах нет ничего непрерывного, в них все конечно, дискретизировано.*

Это очень важный момент, который следует уяснить.
Все вычисления в нем в конечном итоге сводятся к набору простых команд процессора, таких как сложение, вычитание или сравнение с нулем, и практически всегда дают приближенные результаты. Аналоговый же компьютер в состоянии непосредственно оперировать с непрерывными физическими (или какими еще) величинами, но он очень узкоспециализирован, тогда как программируемый цифровой — совершенно *универсален*, и это делает его несравнимо выше.

## 1.4 Конечные автоматы и тьюринг-полные формальные системы

### 1.4.1 Конечные автоматы

*Конечный автомат* есть модель вычислительной системы, последовательно читающей поток символов и переходящей из состояния в состояние. Более формально, автомат $M = (A, Q, q_0, q, F)$, где $A$ — *алфавит*, т.е. множество распознаваемых автоматом символов, $Q$ — множество состояний, в которых может находится автомат, $q_0 \in Q$ — *стартовое состояние*, $q \subset Q$ — множество *конечных состояний*, попадая в любое из которых, автомат завершает свою работу, $F : A \times Q \to Q$ — *функция переходов*, задающая функционирование автомата.
Для примера, рассмотрим конечный автомат, описывающий поведение процесса в некой операционной системе:
Его функция переходов может быть задана следующей матрицей:

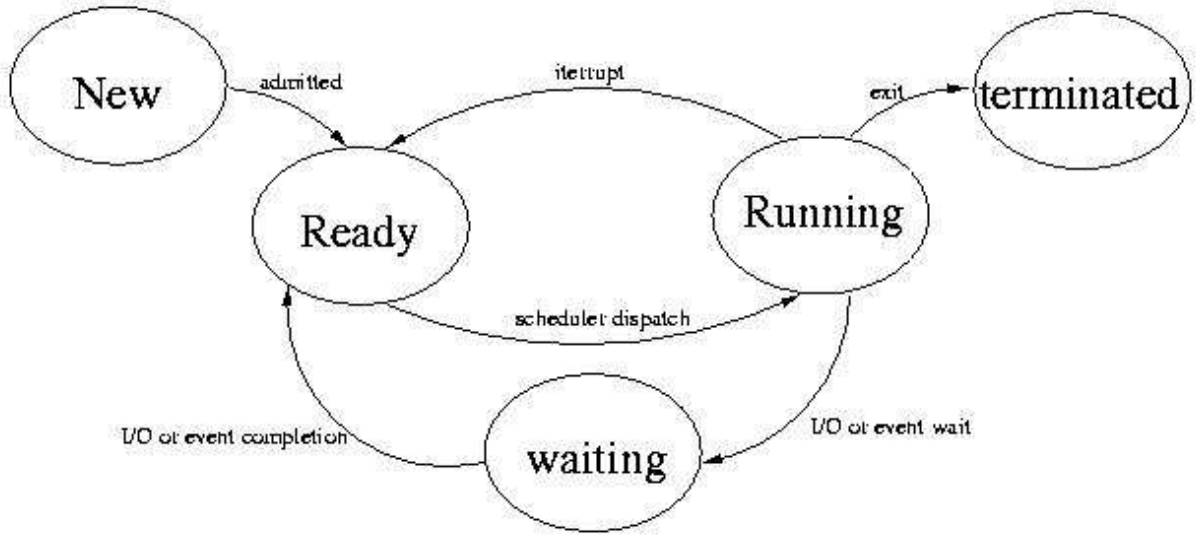


Рис. 1.1: Конечный автомат процесса с пятью состояниями

| | Новый | Ждущий | ЦП | Блокированный | Завершенный |
|---|---|---|---|---|---|
| Запуск | Ждущий | | | | |
| Квант истек | | | Ждущий | | |
| Квант начался | | ЦП | | | |
| Блокировка | | | Блокированный | | |
| Деблокировка | | | | Ждущий | |
| Завершение | | | Завершенный | | |

Таблица 1.6: Функция переходов процесса

### 1.4.2 Тьюринг-полные формальные системы

Конечный автомат может выполнять только ограниченный набор задач, так что последующим шагом может быть его расширение до *машины Тьюринга*, представляющей собой конечный автомат с бесконечной дискретизированной памятью, по которой исполняющий элемент машины (понятно, что это лишь абстракция) может передвигаться в обе стороны и переходить в следующее состояние, соответственно прочитанному символу и текущему состоянию.
*Тезис Черча-Тьюринга* утверждает, что любой алгоритм может быть реализован на машине Тьюринга или любом эквивалентном ей вычислителе при правильном составлении «программы» (т.е. функции переходов), этот тезис, однако, является постулатом, а не доказанной теоремой, поскольку устанавливает связь между формальной конструкцией и интуитивным понятием «алгоритма».
Существует бесконечное множество тьюринг-полных формальных систем, ими является и *лямбда-исчисление* Черча, и *машина Поста*, и практически все языки программирования, включая самые примитивные. Таким образом, понятие тьюринг-полноты имеет в основном теоретическую важность и потому приводится больше ради полноты картины в голове читателя.

## 1.5 Структуры данных

Этот параграф занимает некоторое промежуточное положение между собственно дискретной математикой и компьютерным программированием, между теорией и практикой, однако в силу, во-первых, абстрагированности от отдельных языков или технологий, во-вторых, его «базовости» и рекомендованности к изучению на раннем этапе, стоит рассмотреть его материал сейчас.

Математика оперирует с совершенно абстрактными понятиями вроде множеств или кортежей, необходимость ввода которых обусловлена желанием получения краткой и ясной системы, одновременно и полной, и без ничего лишнего, в полном соответствии с принципом Оккама. Тогда как в программировании есть вторая, огромная и значительная область, отсутствующая в математике — понятие эффективности, разбиваемое на такие составляющие, как размеры используемой памяти, время выполнения и т.п., что связано с тем, что компьютер это, так или иначе, техника. Отсюда следует множество критериев, по которым выбираются и создаются компьютерные *структуры данных*.

Что они из себя представляют в принципе — компьютер можно представлять как определенную иерархию уровней, на каждом из которых есть свой базовый язык и примитивы, с которыми он оперирует. К примеру, на уровне машинного языка такими примитивами есть машинные команды вроде сложения или вызова процедуры, хотя на уровне микроархитектуры процессора — исполнения команды внутри него, аппаратура может и не поддерживать умножение (тогда оно преобразуется в череду сложений встроенной микропрограммой, как во многих CISC-процессорах), в других компьютерах умножение может выполняться непосредственно аппаратурой и т.п. В каждой процессорной архитектуре есть свои базовые типы данных, такие как символы, адреса и целые числа. Аналогичные наборы есть в высокоуровневых языках программирования (но там они обычно сложнее), а переход между этими представлениями реализуется программой-компилятором, превращающей высокоабстрактные понятия в низкоуровневые машинные.

Структуры данных — это составные конструкции, построенные из элементарных, наличие которых позволяет оперировать набором простых сущностей как единым целым. Они могут поддерживаться на уровне языка программирования, на уровне его библиотек, а могут и конструироваться вручную (как оно и есть в реализации библиотеки). К примеру, язык C оперирует теми же сущностями, что и большинство процессоров — целыми и плавающими числами (к целым числам относятся и символы), адресами (тоже целыми числами в общем-то), битовыми полями и т.д., есть также оператор `struct`, позволяющий объединять набор вышеперечисленного (включая сам `struct`), но на том все и закачивается. В C отсутствуют строки или массивы, управляемые как единое целое, для подобных действий используются свои или библиотечные функции. Его высокая эффективность и сравнительная низкоуровневость делают его идеальным языком для действительно «компьютерных» задач вроде написания операционных систем. Во многих других языках,

наоборот, вовсю исполюзуются сложноорганизованные сущности, зачастую рассчитанные на прикладное программирование в определенной предметной области. Я провел общий, довольно паршивый, обзор различий и того, о чем я далее буду говорить, теперь же перейдем собственно к разным структурам данных.

### 1.5.1 Массивы

Одна из самых простых структур данных, индексированная последовательность однотипных элементов.

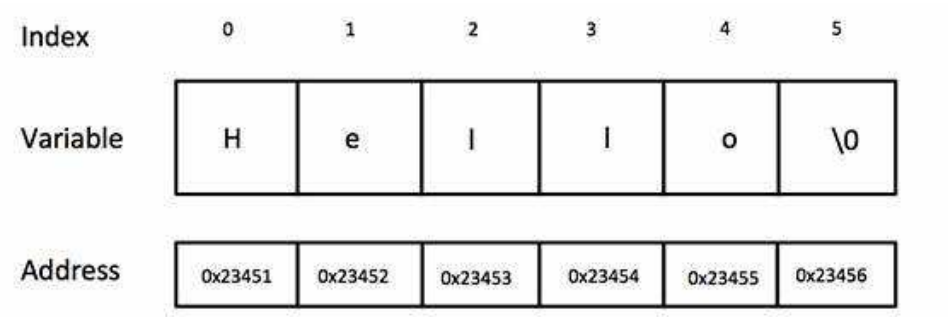


Рис. 1.2: Представление строки как массива символов в C

Некоторые параметры:

- Каждый элемент имеет индекс;
- Поиск по значению в массиве осуществляется перебором элементов либо специальными алгоритмами вроде бинарного деления (если он отсортирован), т.е. за линейное (для многомерного — соответствующая степень) или логарифмическое время (плюс время алгоритма сортировки);
- Поиск по индексу в массиве осуществляется за константное время, прибавлением смещения к адресу первого байта массива;
- Вставка в массив осуществляется путем сдвига элементов, т.е. за линейное время плюс используя дополнительную память;

### 1.5.2 Связные списки

Каждый узел представляется как структура из элементов данных и указателя на следующий (возможно также и на предыдущий) узел, взаимодействие со списком извне происходит через указатель на его первый и/или последний узел (голова и хвост соответственно).

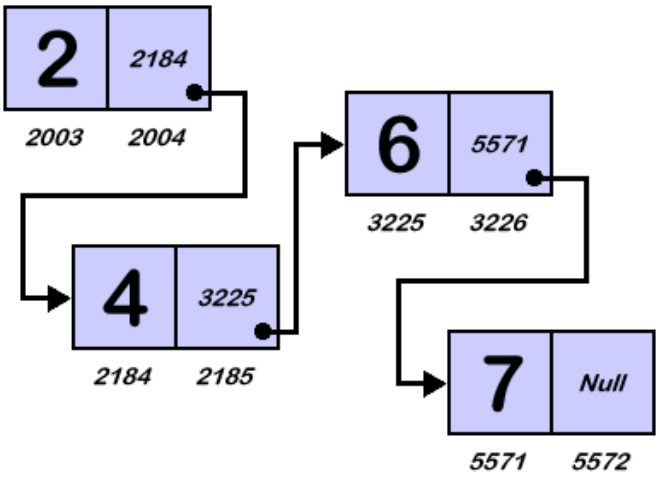


Рис. 1.3: Связный список

Некоторые параметры:

- Элементы не имеют индексов;
- Поиск происходит только по значению и сравнительно долго, перебирая все элементы от головы/хвоста;
- Вставка осуществляется достаточно быстро, без перемещения остальных элементов, путем перезаписи указателей;

### 1.5.3 Деревья

Реализуются подобно связному списку, но в итоге являют собой не последовательность, а связный ациклический граф узлов, у каждого из которых могут быть дочерние узлы. Взаимодействие с внешней программой осуществляется через корень. Наиболее популярная разновидность — бинарное дерево, т.е. каждый узел имеет не более двух дочерних узлов, а узлы расположены согласно некоему порядку сортировки (к примеру, узлы, хранящие строки с меньшими значениями по алфавиту, в левом поддереве, с большими — в правом).

Некоторые параметры бинарных деревьев:

- Элементы не имеют индексов;
- Поиск происходит только по значению и за логарифмическое время, но потребляя приличное количество памяти за счет рекурсивных вызовов в большинстве «древесных» алгоритмов;

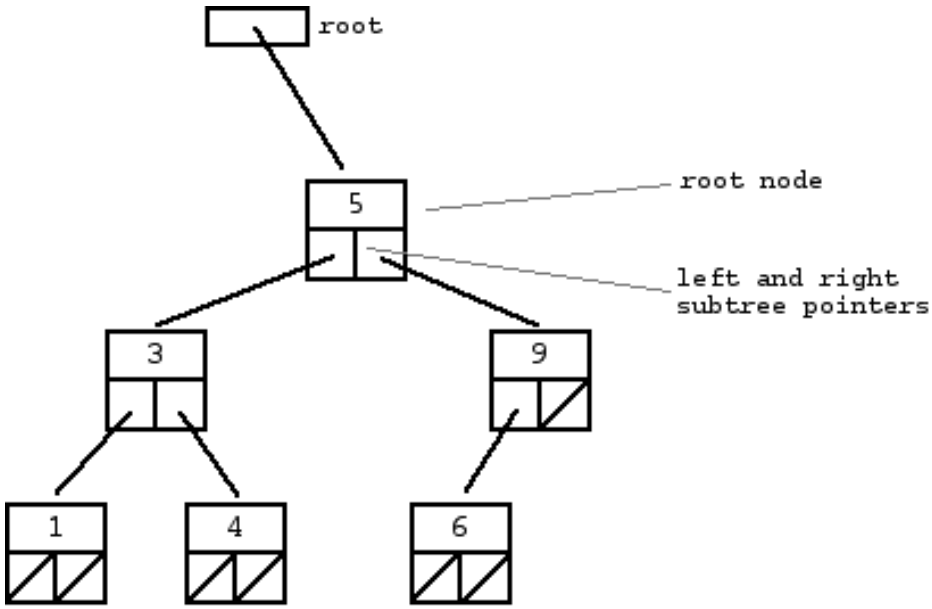


Рис. 1.4: Бинарное дерево

- Вставка происходит за логарифмическое время, но, опять же, с определенными тратами памяти.

### 1.5.4 Хэш-таблицы

Это наиболее удачные с точки зрения скорости поиска структуры, в которых для вставки/поиска какого-то элемента рассчитывается значение *хэш-функции* от ключа (к примеру, если нужно поместить в таблицу структуру со множеством полей, ключем может быть уникальное поле номера и т.п.), а полученный хэш-код используется как индекс, по которому будет помещен элемент. Это может быть индекс в массиве указателей на большие блоки памяти или что-то в этом роде. Соответственно, время поиска элемента будет определяться временем расчета хэш-функции, т.е. будет сравнительно небольшим (константным в идеале), но существует проблема т.н. *хэш-коллизии*, когда хэши разных ключей совпадают и приходится располагать вторые данные рядом с первыми в массиве, на который ссылается индексируемый хэш-кодом указатель, поиск тогда будет все больше вырождаться в линейный. В связи с этим критически важным является пересчет хэшей, сводимый к изменению алгоритма либо разрядности хэш-кодов либо чего-то еще, производимый тогда, когда число блоков данных (или действительных указателей на них) достигнет определенного процента от всего возможного набора (определяемого, например, разрядностью хэш-кода), указывающего на то, что скоро начнутся коллизии и поиск начнет стремиться к линейному. Тогда число хэшей увеличивается (например, в $2^n$ раз при увеличении разрядности на $n$ бит), и момент, когда все они окажутся заняты, отодвигается. Тут важным есть грамотный выбор *fill ratio*, коэффициента, задающего этот процент (зачастую его берут равным 0.75). Чрезмерно малый коэффициент также не годится, поскольку

приведет к постоянному пересчету хэшей и большим временным затратам. Выбор хорошего алгоритма хэширования также, конечно же, играет роль в скорости такой системы.

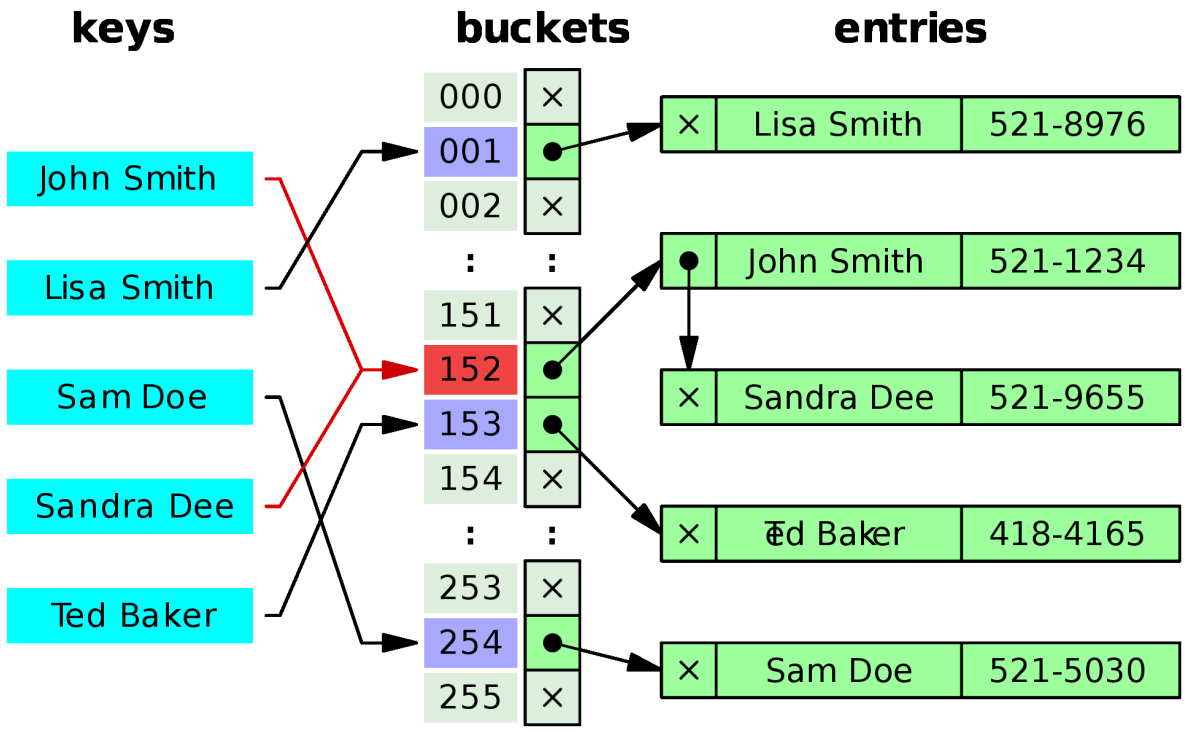


Рис. 1.5: Хэш-таблица и хэш-коллизия

Некоторые параметры:

- Элементы индексируются хэш-кодами;
- Поиск осуществляется (в идеале) за константное время;
- Требуется массив указателей и процедура хэширования;
- Требуется грамотный выбор коэффициента заполнения *fill ratio*, он должен быть не слишком маленький и не слишком большой;

### 1.5.5 Очереди

Очередь — одномерная структура однотипных элементов, подобная массиву (на его основе обычно реализуемая), имеющая два основных указателя — на *голову*

и *хвост* очереди. Посредством первого происходит выборка элементов (он изменяется при извелечении), посредством второго — вставка (изменяется при добавлении). Механизм очереди коротко описывается аббревиатурой FIFO —«первым пришел, первым ушел».

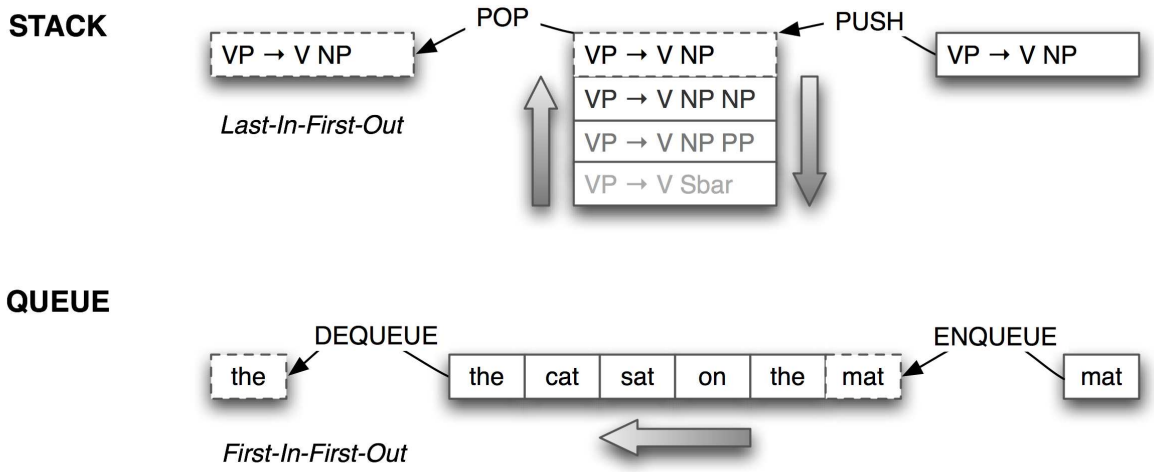


Рис. 1.6: Стек и очередь

Некоторые параметры:

- Поиск происходит за линейное время, перебором элементов;
- Вставка происходит за константное время, добавлением в хвост;
- Извлечение происходит за константное время, выборкой из головы;
- Зачастую нужно осуществлять контроль над попытками извлечь что-то из пустой очереди либо добавить что-то в переполненную;
- Когда голова становится больше хвоста, очередь становится пустой, при этом (или в другое, более подходящее время) следует реинициализировать указатели для оперирования бóльшим количеством памяти;

### 1.5.6 Стеки

Стек — одна из самых важных структур данных, реализуемая на основе того же массива. У него есть всего один указатель на *вершину* стека, через который происходит и вставка, и извлечение элементов. При добавлении элемента он помещается на вершину стека, указатель при этом увеличивается, при извлечении —убирается из вершины (указатель, соответственно, уменьшается). Механизм стека коротко описывается аббревиатурой LIFO —«последним пришел,

первым ушел».

Некоторые параметры:

- Поиск происходит за линейное время, перебором элементов;
- Вставка происходит за константное время, добавлением в вершину;
- Извлечение происходит за константное время, выборкой из вершины;
- Зачастую нужно осуществлять контроль за попытками извлечь что-то из пустого стека или добавить что-то в переполненный;

## 1.6 Вопросы

*Примечание:* составляя вопросы, я отказался от концепции тестов, поскольку у многих они часто порождают немного неправильный мыслительный подход, рассчитанный на использование приемов вроде метода исключения или анализа частоты выпадания определенной буквы в ответах, т.е. побуждают для получения ответа мыслить о чем-то другом, не касающемся собственно предмета. Помимо этого, они обычно очень поверхностны. Так что вопросы к этой и следующим главам рассчитаны одновременно на глубокое понимание концепций и умение грамотно реализовать их на практике (собственно, без одного нет и другого, не так ли?).

1. Чему равно десятичное значение `float`-числа `0xC378C000`?
2. Программа `od` в UNIX-системах выводит bin/oct/dec/hex-представление входного потока байт строками по 16 байт, разбивая его на слова заданной длины с заданным порядком байт в пределах каждого слова (для более подробной информации обратитесь к man-странице этой программы: `$ man od`). Поэкспериментируйте с этой программой, проверьте свое владение различными системами счисления — вы должны щелкать их как семечки!
3. Составьте ДНФ и упростите следующую функцию: $Y = A!B + B!C + C!D + D!E + E!A$. Для упрощения используйте законы булевой алгебры и/или любой известный вам алгоритм, в случае нескольких подходов сравните результаты.
4. Чему равен булеан множества $S = \{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}, \{\emptyset, \{\emptyset\}, \{\{\emptyset\}\}, \{\emptyset, \{\emptyset\}\}\}\}$? Задайте S при помощи закона, а не перечислением элементов. Найдите булеаны следующих трех множеств после S, сформированных по тому же принципу. Поймите, насколько это муторно, и напишите программу генерации булеана заданного множества (желательно на C и без специальных библиотек).

5. Функции высших порядков — это функции, принимающие функции в качестве аргументов. Каррирование — это преобразование функции нескольких аргументов в функцию высшего порядка, его можно задать как $K : (A \times B \to C) \to (A \to (B \to C))$. В таких формальных системах, где не существует декартовых степеней множеств $> 1$, каррирование позволяет представить соответствующие функции как функции высших порядков. Поиграйте с этими возможностями, написав программу на C (в языках со средствами функционального программирования, таких как Lisp или Haskell, функции высшего порядка являются частью языка, потому там это значительно проще), в которой вместо функций нескольких аргументов использовались бы функции, принимающие один аргумент, обрабатывающие его и возвращающие указатель на функцию, принимающую следующий и т.п.

6. Напишите программу (на C или ассемблере), моделирующую поведение машины Тьюринга или машины Поста.

7. Изучите формальную систему *лямбда-исчисление*, также являющуюся тьюринг-полной. Опишите суть *ленивых вычислений*.

8. Реализуйте алгоритмы работы с упорядоченным бинарным деревом — вставки/поиска элемента, обхода дерева с выводом его содержимого на терминал или в файл.

9. *Обратная польская запись* — нотация, в которой оператор ставится после операнда/операндов, что устраняет неоднозначности приоритета и избавляет от необходимости использовать скобки. Напишите программу преобразования обычной *инфиксной* записи (во входном выражении скобки расставляются где возможно, либо же список приоритетов операций хранится в памяти или файле) в постфиксную.

10. Пользуясь результатами предыдущего задания, напишите стековый калькулятор, работающий по принципам обратной польской записи (программный вариант выпускавшихся когда-то калькуляторов Hewlett-Packard).

11. Реализуйте хэш-таблицы и практическое использование их в некой программе. Это может быть фрагмент препроцессора или транслятора, хранящий символьные константы для последующей замены, некий словарь или что-то другое. Поэкспериментируйте с различными значениями коэффициентов заполнения.

12. Реализуйте статическую (`.a`) или динамическую (`.so`) библиотеку работы со сложными структурами данных (пользуясь предыдущими наработками) — массивами, стеками, очередями, списками, деревьями, хэш-таблицами и т.п. Поместите ее заголовочные `.h`-файлы в каталог `/usr/include`, а бинарники — в `/usr/lib` (проверьте разрядность).